References

Superoptimization, Program Synthesis, and Learned Cost Models — Reading List for the Regression-Guided RISC-V Superoptimizer Project

---

1. Foundational and Classical Superoptimization

[1] H. Massalin, "Superoptimizer: A Look at the Smallest Program," Proceedings of ASPLOS, 1987. The original superoptimizer: brute-force enumeration of short instruction sequences checked for equivalence against a target.

[2] S. Bansal and A. Aiken, "Automatic Generation of Peephole Superoptimizers," Proceedings of ASPLOS, 2006. Enumerative approach building a reusable database of provably optimal short sequences.

[3] R. Joshi, G. Nelson, and K. Randall, "Denali: A Goal-Directed Superoptimizer," Proceedings of PLDI, 2002. Uses a theorem prover and equality saturation to search for optimal code, an early alternative to pure enumeration.

2. Stochastic and Search-Based Superoptimization

[4] E. Schkufza, R. Sharma, and A. Aiken, "Stochastic Superoptimization," Proceedings of ASPLOS, 2013 (arXiv:1211.0557). Introduces STOKE: MCMC-based stochastic search over loop-free binaries, using a cost function balancing correctness and performance.

[5] E. Schkufza, R. Sharma, and A. Aiken, "Stochastic Optimization of Floating-Point Programs with Tunable Precision," Proceedings of PLDI, 2014. Extends the STOKE approach to floating-point code and precision/performance trade-offs — useful contrast material given this project excludes floats.

[6] P. M. Phothilimthana, A. Thakur, R. Bodik, and D. Dhurjati, "GreenThumb: Superoptimizer Construction Framework," UC Berkeley Technical Report UCB/EECS-2016-8, 2016. A retargetable framework combining enumerative, stochastic, and SMT-based search across different ISAs.

[7] P. M. Phothilimthana, A. Thakur, R. Bodik, and D. Dhurjati, "Scaling Up Superoptimization," Proceedings of ASPLOS, 2016. Companion work on making hybrid-search superoptimization scale to larger programs and ISAs.

3. Program Synthesis and Equivalence Checking

[8] PrediPrune: Reducing Verification Overhead in Souper with Machine Learning Driven Pruning, arXiv:2509.16497, 2025. Describes SMT-based equivalence checking in the Souper superoptimizer and proposes an ML-driven pruning step to reduce solver calls — directly relevant to speeding up verification in this project.

[9] epex-formal-rv32-model: A formal 32-bit RISC-V model for instruction-level program synthesis using an SMT solver (open-source project, ICS-JKU). Demonstrates SMT-based synthesis directly on RISC-V register/PC states, a close structural match to this project's target ISA.

[10] B. Churchill, O. Padon, R. Sharma, and A. Aiken, "Semantic Program Alignment for Equivalence Checking," Proceedings of PLDI, 2019. Techniques for aligning and comparing two programs' semantics efficiently, relevant background for building a robust equivalence checker.

[11] R. Alur et al., "Syntax-Guided Synthesis (SyGuS)," Formal Methods in Computer-Aided Design (FMCAD), 2013. Establishes the general syntax-guided program synthesis framework that much SMT-based superoptimization work (including Souper-style approaches) builds on.

4. Learned / Regression-Based Cost Models

[12] C. Mendis, A. Renda, S. Amarasinghe, and M. Carbin, "Ithemal: Accurate, Portable and Fast Basic Block Throughput Estimation using Deep Neural Networks," Proceedings of ICML, 2019 (arXiv:1808.07412). Frames throughput estimation as a regression problem over instruction sequences, cutting mean error by more than half versus analytical tools such as llvm-mca and IACA — the closest prior example of the regression cost model this project proposes, though at a much larger (x86-64, neural) scale.

[13] O. Sykora et al., "GRANITE: A Graph Neural Network Model for Basic Block Throughput Estimation," arXiv:2210.03894, 2022. Improves on Ithemal by representing a basic block as a data-dependency graph rather than a flat sequence before regression — relevant to this project's dependency-depth feature choice.

[14] COMET: A Neural Cost Model Explanation Framework, arXiv:2302.06836, 2024 (MLSys). Surveys and compares learned cost models (e.g. Ithemal) against simulation-based analytical models (llvm-mca, uiCA), directly motivating the accuracy/speed trade-off this project's regression model must navigate.

[15] S. Kaufman et al., "A Learned Performance Model for Tensor Processing Units," Proceedings of MLSys, 2021 (arXiv:2008.01040). Learned regression-style performance model guiding compiler decisions for TPU kernels — an example of the same learned-cost-model pattern applied outside the instruction level.

[16] ML-driven Hardware Cost Model for MLIR, arXiv:2302.11405, 2023. Surveys ML-based hardware cost modeling approaches (including Ithemal and TPU performance models) and proposes one for MLIR-level code, useful as a broader picture of where learned cost models fit in compiler tooling.

[17] Z. Chen et al., "Ansor: Generating High-Performance Tensor Programs for Deep Learning," Proceedings of OSDI, 2020 (arXiv:2006.06762). Uses a learned cost model to rank candidate program schedules during search instead of measuring each one directly — the same search-loop integration pattern this project intends to use with its regression model.

5. Learning- and RL-Guided Superoptimization (Recent Work)

[18] Learning Guided Enumerative Synthesis for Superoptimization, in Model Checking Software (SPIN), Springer, with continued citing work through 2025 including "Neural-Guided Superoptimization in Ethereum," Information and Software Technology, vol. 186, 2025. Uses a learned model to prioritize which candidates an enumerative synthesizer explores, closely related to how this project's regression model is meant to guide search.

[19] SuperCoder: Assembly Program Superoptimization with Large Language Models, arXiv:2505.11480, 2025. Trains an LLM via reinforcement learning (PPO and GRPO) with a reward combining correctness and measured speedup, applied directly to assembly-level superoptimization — representative of the current frontier, useful as contrast to this project's simpler, more tractable regression approach.

[20] Pearl: Automatic Code Optimization using Deep Reinforcement Learning, Proceedings of the 39th ACM International Conference on Supercomputing (ICS), 2025. Applies deep RL to automatic code optimization, illustrating an alternative to supervised regression for the search-guidance model — worth noting as a possible stretch-goal extension.

[21] E. Albert, M. Garcia de la Banda, A. Hernández-Cerezo, A. Ignatiev, A. Rubio, and P. Stuckey, "SuperStack: Superoptimization of Stack-Bytecode via Greedy, Constraint-Based, and SAT Techniques," Proceedings of the ACM on Programming Languages (PLDI), 2024. Demonstrates constraint- and SAT-based superoptimization on a different (stack-based bytecode) target, useful for comparing search strategies against the register-based RISC-V setting.

6. RISC-V and Machine Learning Context

[22] RISC-V and Machine Learning: A Survey, The Journal of Supercomputing, 2026. Broad survey of ML applied to and on RISC-V hardware/tooling, useful for framing why RISC-V is an active target and for pointing to available simulators and toolchains for the implementation phase.

[23] A. Waterman and K. Asanovi■ (eds.), "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," RISC-V Foundation. The authoritative specification for RV32I and its extensions (M, F, D); the primary reference for precisely defining the reduced instruction subset used in this project.

Note: entries are grouped thematically rather than strictly chronologically to mirror how they will be used in the project's literature review (Section 4 of the project draft). Full bibliographic details (volume/page numbers where applicable) should be verified against the publisher's record before final submission.